

[Edit](#)

Write Unit Tests Automatically by Using Copilot

Challenge Overview

Understand the scenario

You are a Developer at Hexelo, a company that needs to efficiently write and test Python code.

In this Challenge Lab, you will write unit tests automatically using Copilot. First, you will configure Visual Studio Code to support Python development. Next, you will set up GitHub Copilot to assist in coding. Then, you will create and test a simple Python script. Finally, you will implement unit tests using GitHub Copilot to ensure your code is reliable.

 You must complete the **Set Up Git and Connect to GitHub** setup lab before starting any challenge in this series. The set-up lab should only be completed once.

Navigating the Challenge Lab

Unable to process content from <https://raw.githubusercontent.com/LODSContent/Challenge-V3-Framework/main/Templates/Environments/None.md>

 ► Quick tips for navigating the Challenge Lab instructions.

[New to Challenge Labs? Click here to learn more.](#)

Configure Visual Studio Code and install the Python extension

Hints Enabled

No Yes

 You must complete the **Set Up Git and Connect to GitHub** setup lab before starting any challenge in this series. The set-up lab should only be completed once.

- Sign in to the virtual machine as **LabUser** by using  **Passw0rd!** as the password.
- Enter your GitHub Username, Email Address, and Password into the following associated textboxes:

GitHub Username

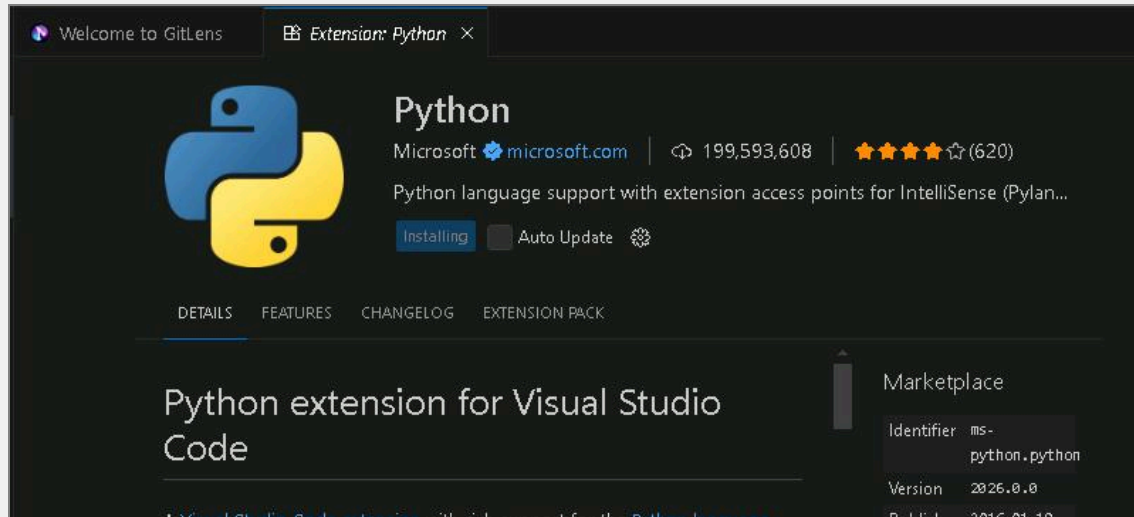
GitHub Email Address

GitHub Password

- Install the  **Python** extension from **Microsoft** for  **Visual Studio Code**.

💡 Expand this hint for guidance on installing the Python extension.

- On the Windows taskbar, in Search, enter **T** Visual Studio Code, and select it to open **VS Code**.
- Select the Extensions icon in the Activity Bar on the side of the window.
- In the Extensions view, search for **T** Python.
- Install the extension published by **Microsoft**.



💡 Want to learn more? Review the documentation on [Using Python in VS Code](#).

❓ Make sure to reload **Visual Studio Code** after installing the extension to activate it.

- Sign in to GitHub to authenticate Copilot usage in **Visual Studio Code** using **T** <Name> as the username and **T** <GithubPW> as the password.

💡 Expand this hint for guidance on signing in to GitHub for Copilot.

- In **Visual Studio Code**, in the bottom right-hand corner, select **Signed out**, and then select **Sign in to use Copilot**.
- On the pop-up, select **Sign in**, and then enter **T** <Name> as the username and **T** <GithubPW> as the password.
- Select **Continue** and then **Open** to authorize **Visual Studio Code** to use GitHub Copilot.

💡 Want to learn more? Review the documentation on using a [GitHub account with Copilot](#).

- Configure **Visual Studio Code** to use the default Python interpreter on your system.

💡 Expand this hint for guidance on configuring the Python interpreter. ^

- In **Visual Studio Code**, open the **Command Palette** by selecting the **View** menu and choosing **Command Palette**.
- Enter **T** **Python: Select Interpreter**.
- From the list, choose the default Python interpreter installed on your system.

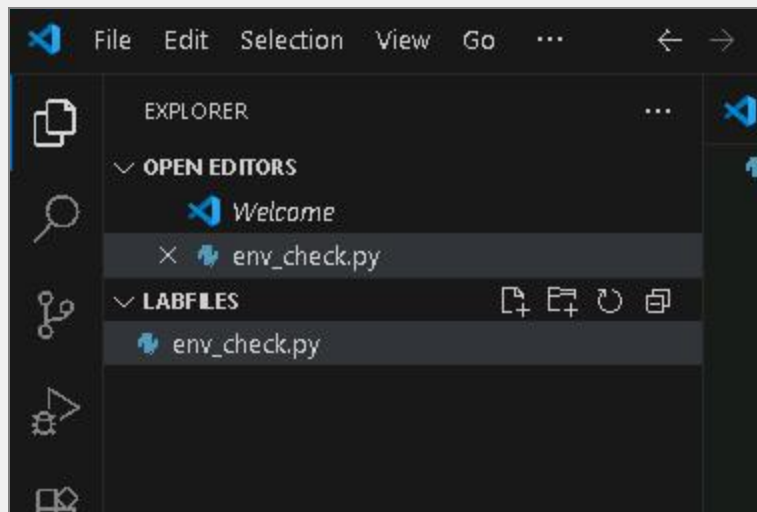
💡 Want to learn more? Review the documentation on [Selecting Interpreters](#).

📄 You may close the *Welcome* windows at any time.

- In **Visual Studio Code**, create a new Python file named **T** **env_check.py** in the **T** **D:\LabFiles** folder, and then confirm that the file is using the **Python** language mode.

💡 Expand this hint for guidance on creating the env_check.py file. ^

- In **VS Code**, on the menu bar, select **File**, and then select **Open Folder**.
- Navigate to the **T** **D:\LabFiles** folder and select **Select Folder**.
- On the *Do you trust the authors of the files in this folder?* dialog, select **Yes, I trust the authors**.
- In Explorer, on the **D:\LabFiles** folder, select **New File**.
- Name the file **T** **env_check.py**, and then press **Enter**.



- At the top of `T` `env_check.py`, paste the following GitHub Copilot prompt comment that describes the script you want Copilot to generate:

python

```
# GitHub Copilot, create a simple Python script to verify that Visual Studi
# and the Python extension are installed and working correctly.
#
# Requirements:
# - Define a function called `get_environment_info()` that returns:
#   - The current Python version string.
#   - The full path to the Python executable.
# - In a `main()` function:
#   - Call `get_environment_info()`.
#   - Print a friendly message like: "VS Code and Python are configured cor
#   - Print the Python version and executable path on separate lines.
# - Add the standard `if __name__ == "__main__":` guard and call `main()`.
# - Include basic inline comments explaining what each function does.
#
# Please write clean, beginner-friendly code.
```

💡 Expand this hint for guidance on adding the Copilot prompt comment. ^

- In the `T` `env_check.py` file, move the cursor to the first line.
- Copy and then paste the supplied comment block into the file.

- Save the `env_check.py` file.

💡 Expand this hint for guidance on saving a file. ^

- Select the **File** menu, then select **Save** or press **Ctrl+S**.
- Ensure the `env_check.py` file is saved in the **D:\LabFiles** directory.

- Use GitHub Copilot to generate the `T` `env_check.py` script based on the prompt comment, and then review and accept or refine the suggested code.

💡 Expand this hint for guidance on generating code with GitHub Copilot.

- Place the cursor on a new line below the comment block.
- Enter `T` `def` and wait for GitHub Copilot to display a suggested implementation for the `T` `get_environment_info` function.
- Accept Copilot's suggestion by pressing **Tab** or **Enter**.
- The generated code should look similar to the screenshot below:

```

14 #
15 # Please write clean, beginner-friendly code.
16
17 v def get_environment_info():
18 v     """
19     This function retrieves the current Python version and the path to the Python exe
20     Returns:
21     | tuple: A tuple containing the Python version string and the executable path.
22     """
23     import sys
24     python_version = sys.version
25     python_executable = sys.executable
26     return python_version, python_executable

```

- **Save** the updated `T` `env_check.py` file.

- At the bottom of the `env_check.py` file, paste the following code, and then save the file:

```

def main():
    """Entry point that prints a confirmation message and environment detail
    python_version, python_executable = get_environment_info()
    print("VS Code and Python are configured correctly!")
    print(f"Python version: {python_version}")
    print(f"Python executable: {python_executable}")

if __name__ == "__main__":
    main()

```

📄 The `env_check.py` file should look similar to the following screenshot:

```
def get_environment_info():
    """
    This function retrieves the current Python version and the path to the Python executable.
    Returns:
    |   tuple: A tuple containing the Python version string and the executable path.
    """
    import sys
    python_version = sys.version
    python_executable = sys.executable
    return python_version, python_executable

def main():
    """Entry point that prints a confirmation message and environment details."""
    python_version, python_executable = get_environment_info()
    print("VS Code and Python are configured correctly!")
    print(f"Python version: {python_version}")
    print(f"Python executable: {python_executable}")

if __name__ == "__main__":
    main()
```

- Run the `env_check.py` script in a terminal to verify that **Visual Studio Code** and the Python extension are configured correctly.

💡 Expand this hint for guidance on running the `env_check.py` script. ^

- In **Visual Studio Code**, open a new terminal by selecting **Terminal** > **New Terminal**.
 - Confirm that the terminal's current directory is the folder that contains `env_check.py`.
 - If it is not, change to that directory by using the `cd D:\LabFiles` command.
- In the **VS Code** Terminal, run the script by entering the following command: `python env_check.py`.
- Review the output in the terminal and confirm that it:
 - Displays a success message indicating that **VS Code** and Python are configured correctly.
 - Shows the current Python version.
 - Shows the path to the Python executable.
- If the script runs without errors and prints the expected information, your environment verification is complete.

```
PS D:\LabFiles> python env_check.py
VS Code and Python are configured correctly!
Python version: 3.12.10 (tags/v3.12.10:0cc8128, Apr 8 2025, 12:21:36) [MSC v.1943 64 bit (AMD64)]
Python executable: C:\Users\LabUser\AppData\Local\Programs\Python\Python312\python.exe
PS D:\LabFiles>
```

💡 Want to learn more? Review the documentation on [working with Python Files](#).

⚠️ Configuration errors may indicate missing or incorrect Python installation paths. Check your settings for accuracy.

Check your work

Verify

Set up GitHub Copilot

Hints Enabled

No Yes

- If necessary, enable Copilot in **VS Code** by installing its extension.

💡 Expand this hint for guidance on enabling Copilot.

- In **Visual Studio Code**, navigate to the **Extensions** view.
- Search for  **GitHub Copilot** and install the extension.



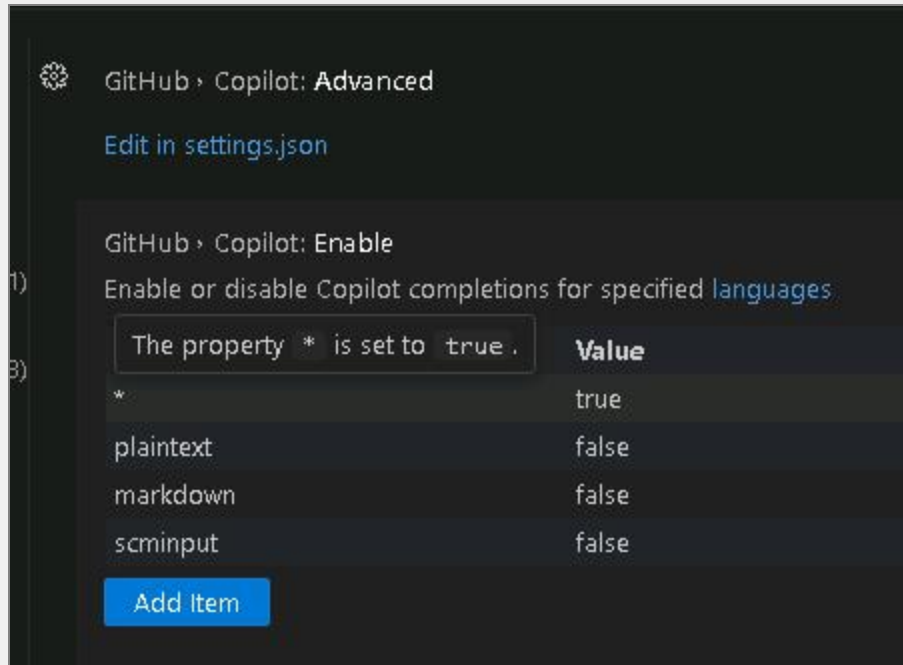
💡 Want to learn more? Review the documentation on [GitHub Copilot Installation](#).

📄 Enabling Copilot requires an active GitHub account with beta access to Copilot.

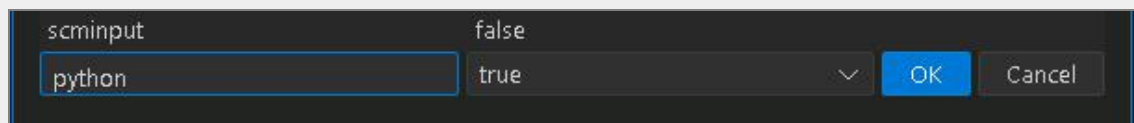
- Adjust Copilot settings so that code completions are enabled for Python, but minimized for non-code content like markdown, which supports a test-driven development workflow.

💡 Expand this hint for guidance on adjusting Copilot settings.

- In **Visual Studio Code**, in the lower-left corner, select the **gear** icon, and then select **Settings**.
- In the **Settings** search box at the top, enter **GitHub Copilot**.
- Under **GitHub > Copilot: Enable**, review the language list.
- Ensure the row with ***** is set to **true** so Copilot is enabled by default.



- Add an entry for **python** (if it does not already exist) and set it to **true** so code completions are always available in Python files.



- Verify that **plaintext**, **markdown**, and other non-code languages you use for lab instructions are set to **false** so Copilot does not rewrite your written content.

- Close the **Settings** tab.

📄 Your Copilot configuration is now optimized for writing Python tests and code while avoiding unwanted suggestions in documentation or lab markdown files.

⚠ Ensure that Copilot's context and suggestions align with your development requirements to prevent irrelevant code recommendations. Consider customizing settings frequently based on project needs.

Check your work

Verify

Create initial Python script

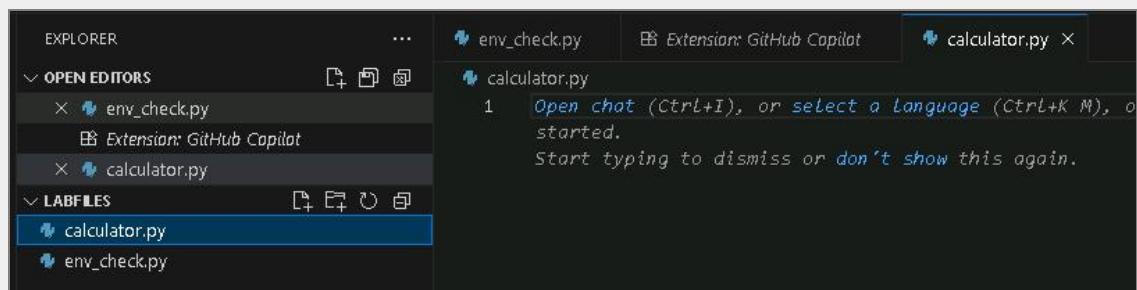
Hints Enabled

No Yes

- Create a new file named `calculator.py` in the `D:\LabFiles` directory.

💡 Expand this hint for guidance on creating a new file.

- In **Visual Studio Code**, select the **File** menu, then select **New File**.
- **Save** the new file as `calculator.py` in the `D:\LabFiles` directory.



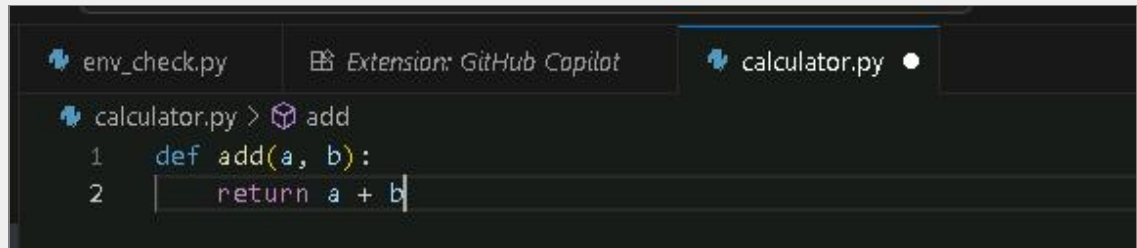
💡 Want to learn more? Review the documentation on [Creating Files in VS Code](#).

- Implement a simple `add` function by using `add(a, b)`, that returns the sum of two numbers.

💡 Expand this hint for guidance on implementing a function. ^

- In **Visual Studio Code**, open the `calculator.py` file.
- In **calculator.py**, define a new function by using `def add(a, b):`.
- Use a return statement to calculate and return the sum by using: `return a + b`.

Note: Ensure the return statement is indented below the definition for the add function.



```
env_check.py  Extension: GitHub Copilot  calculator.py •
calculator.py > add
1  def add(a, b):
2      return a + b
```

💡 Want to learn more? Review the documentation on [Writing Python Functions](#).

- Save the **calculator.py** file in the **D:\LabFiles** directory.

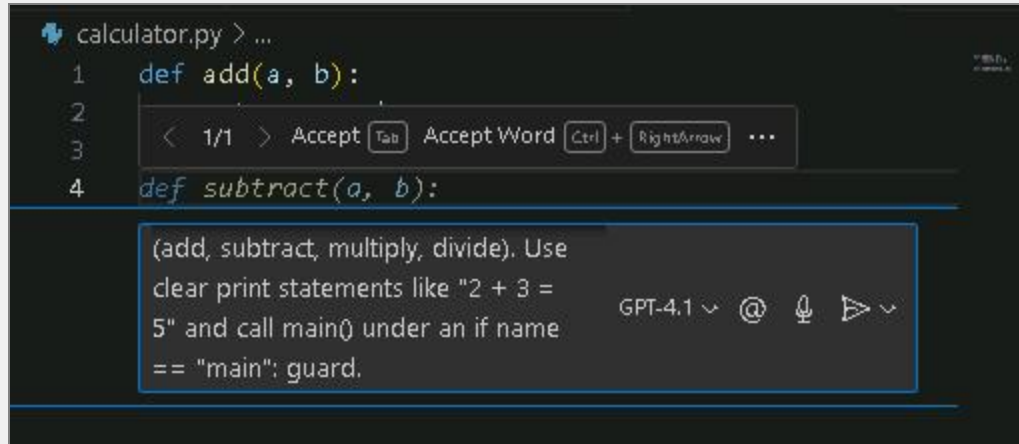
📄 Saving your file frequently can prevent data loss.

- Use the GitHub Copilot Chat to add sample calculations to the `calculator.py` file by using the following prompt:

📄 In calculator.py, add a main() function that prints example calculations for each operation (add, subtract, multiply, divide). Use clear print statements like "2 + 3 = 5" and call main() under an if `__name__ == "__main__":` guard.

💡 Expand this hint for guidance on using Copilot Chat to add sample calculations. ^

- In the **calculator.py** script file, select **Ctrl+I** to open the Copilot Inline Chat editor.
- In the chat box, enter the prompt from above and press **Enter** to send it.

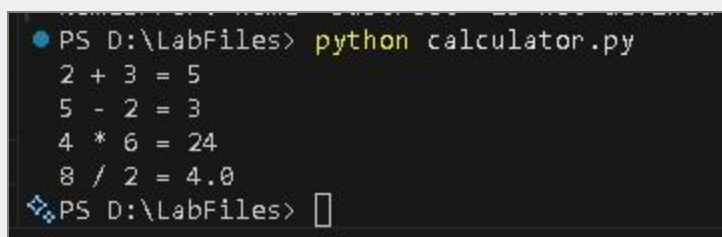


- Review Copilot's proposed changes and accept them to update the **calculator.py** file.

💡 Want to learn more? Review the documentation on [Running Python Code](#).

- Save the **calculator.py** file and run `python calculator.py` in the integrated terminal to confirm that the sample calculations are printed.

📄 Your output should look like the following screenshot:

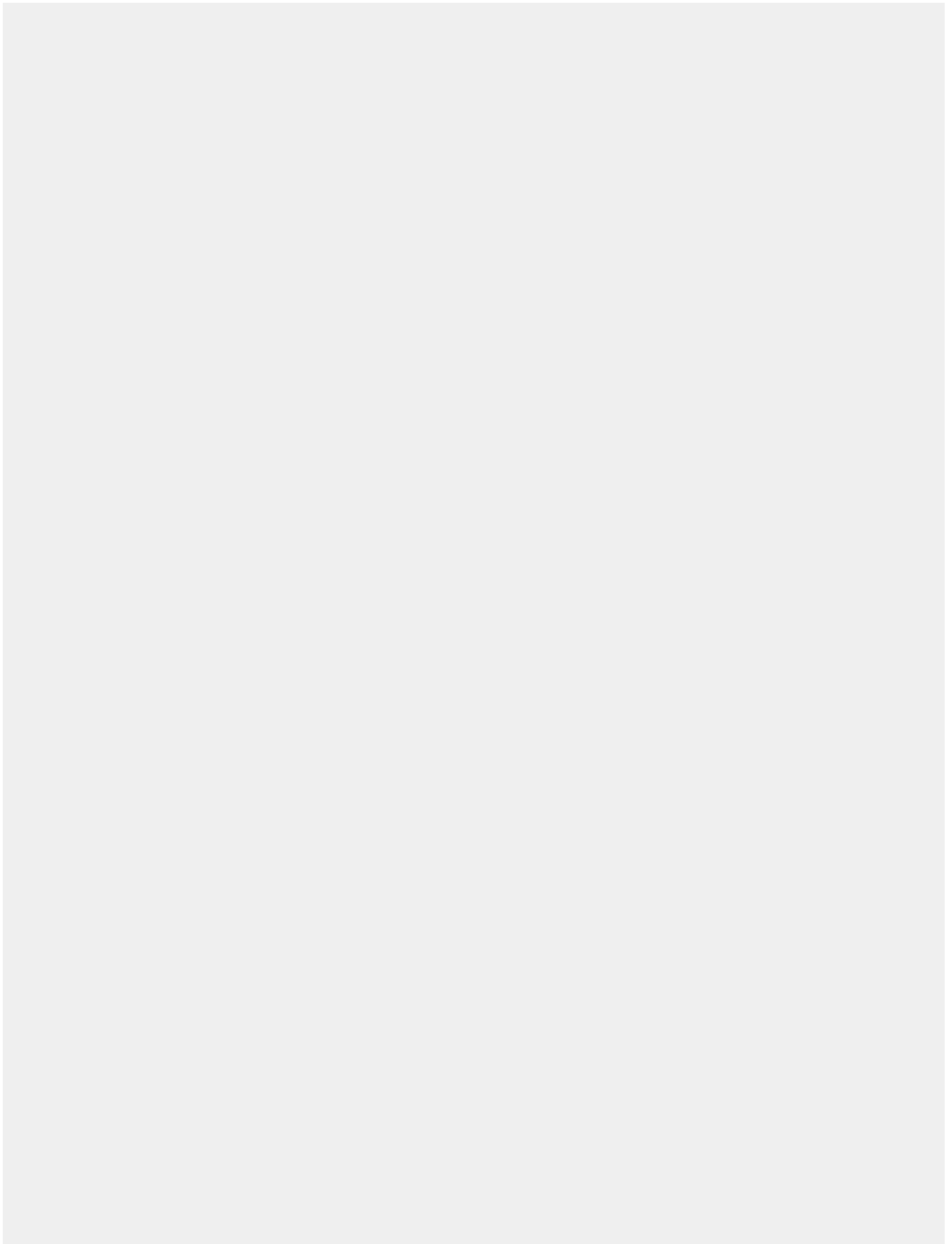


⚠️ Ensure Python is properly installed and added to your system's PATH to run scripts effortlessly.

In case of persistent issues, verify the PATH configuration or interpreter selection.

Check your work

Verify



Implement unit tests using Copilot

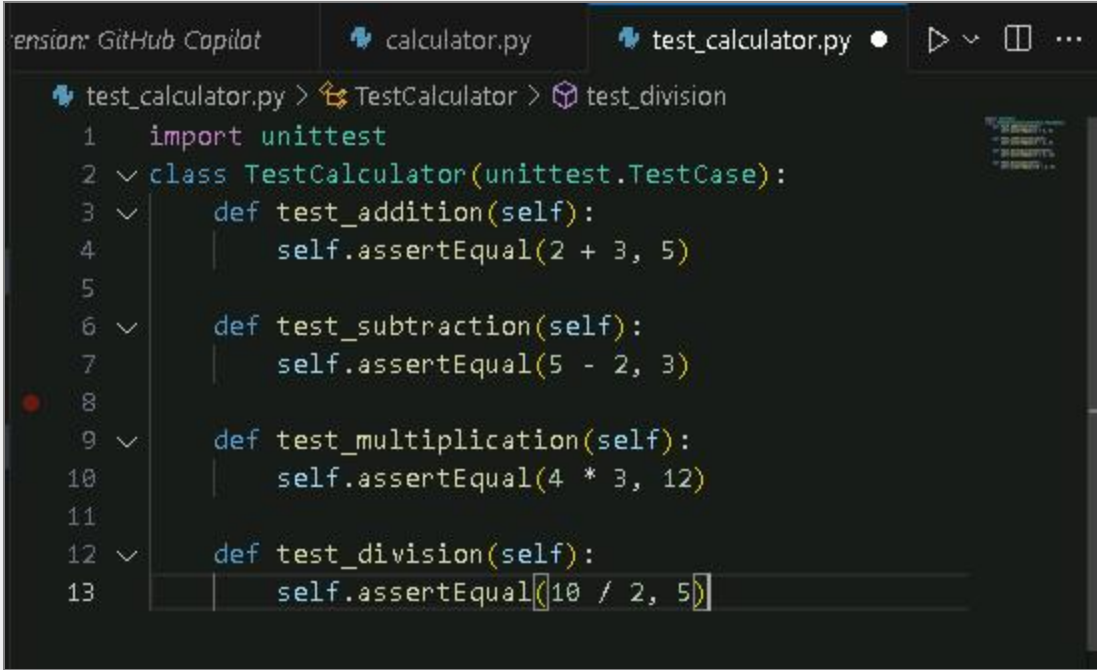
Hints Enabled

No Yes

- Create a new file named `test_calculator.py` in the `D:\LabFiles` directory.
- Write a unit test for the `add` function by using the `unittest` framework with Copilot suggestions.

 Expand this hint for guidance on writing a unit test. 

- In the `test_calculator.py` file, enter `import unittest`, and then press **Enter**.
- On the next line, enter `class TestCalculator(`.
- Copilot will usually suggest the entire test scaffold.
- Select **Tab** to accept Copilot's suggestion.



```
test_calculator.py > TestCalculator > test_division
1 import unittest
2 class TestCalculator(unittest.TestCase):
3     def test_addition(self):
4         self.assertEqual(2 + 3, 5)
5
6     def test_subtraction(self):
7         self.assertEqual(5 - 2, 3)
8
9     def test_multiplication(self):
10        self.assertEqual(4 * 3, 12)
11
12    def test_division(self):
13        self.assertEqual(10 / 2, 5)
```

 Want to learn more? Review the documentation on [Writing Unit Tests in Python](https://docs.python.org/3/library/unittest.html).

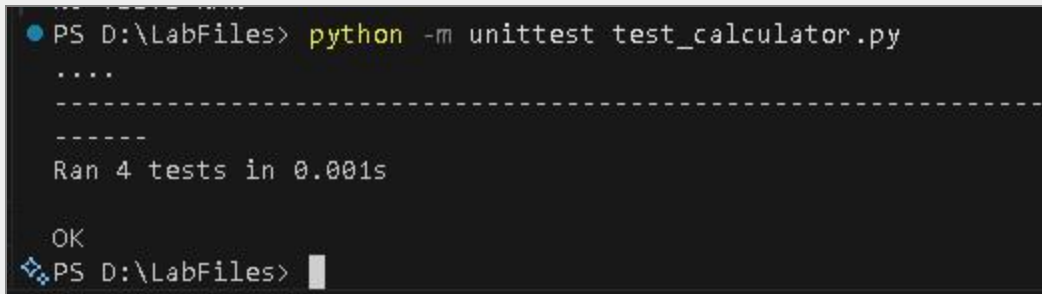
- Save the **test_calculator.py** file.
- Execute the tests in **VS Code** using the built-in terminal.

💡 Expand this hint for guidance on executing tests. ^

- Open the **VS Code** Terminal and run the following command:

📄 `python -m unittest test_calculator.py`

- Review test results in the terminal output.



```
PS D:\LabFiles> python -m unittest test_calculator.py
.....
-----
-----
Ran 4 tests in 0.001s

OK
PS D:\LabFiles>
```

💡 Want to learn more? Review the documentation on [Running Unit Tests](#).

- Confirm all tests pass successfully.

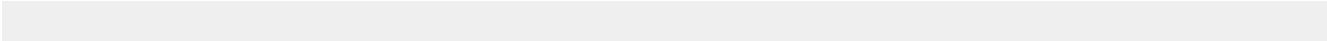
💡 Expand this hint for guidance on confirming test success. ^

- Check the terminal output for the amount of **4 tests** and the message **OK**, indicating all tests passed.
- Investigate any test failures by reviewing the error messages and the test code.

💡 Want to learn more? Review the documentation on [Interpreting Test Results](#).

⚠️ Be attentive to failure messages, which can indicate unexpected behavior in your function.
In case of failures, revise the logic in both the *add* function and its tests carefully.

Check your work


Verify

Summary

Congratulations, you have completed the **Write Unit Tests Automatically by Using Copilot** Challenge Lab.

You have accomplished the following:

- Configured Visual Studio Code and installed the Python extension.
- Set up GitHub Copilot.
- Created initial Python script.
- Implemented unit tests using Copilot.

Ending your lab

To ensure your lab is recorded as complete, select **Submit** or **End** below. Exiting [X] the lab will result in an incomplete status.

 Once you select **Submit** or **End**, you will not be able to return to this Challenge Lab.

Your feedback is important!

As you end your Challenge Lab, please take a few minutes to complete the short survey that will appear in the next window. Alternatively, you may provide your feedback directly to [Challenge Labs feedback](#).

Looking for your next Challenge Lab?

Below are recommended Challenge Labs to try next.

- Set Up Git and Connect to GitHub [Prerequisite Lab]
- Create a Class from a Prompt Using GitHub Copilot [Guided]
- Build Features by Using Copilot in Python [Guided]
- Write Unit Tests Automatically by Using Copilot [Guided]
- Add Comments and Docstrings by Using Copilot [Guided]
- Develop an API Client with GitHub Copilot [Guided]

- Refactor Python Code by Using Copilot [Guided]
- Create Shell Scripts by Using Copilot [Guided]
- Automate CI Workflows by Using GitHub Actions [Guided]
- Document Projects by Using Markdown and Copilot [Guided]
- Refactor and Optimize Code with GitHub Copilot [Guided]
- Build Command-Line Tools by Using Copilot [Guided]
- Utilize Copilot for Python Code Correction [Guided]

